

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- ✓ Cette épreuve est composée d'un exercice et de deux parties tous indépendants ;
- ✓ Toutes les instructions et les fonctions demandées seront écrites en Python ;
- ✓ Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- ✓ Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

Important : Le candidat doit impérativement commencer par traiter toutes les questions de l'exercice ci-dessous, et écrire les réponses dans les premières pages du cahier de réponses.

Exercice : (4 points)

La suite de **Fibonacci** (F_n) est définie par : $F_0 = 1$, $F_1 = 1$ et $F_n = F_{n-2} + F_{n-1}$ pour $n > 1$.

Pour calculer la valeur du terme F_n de la suite Fibonacci, on utilise l'algorithme suivant :

Fonction fibo (n) :

$a \leftarrow 1$

$b \leftarrow 1$

Pour $i = 2$ à n faire

$c \leftarrow a + b$

$a \leftarrow b$

$b \leftarrow c$

Fin Pour

Retourner b

Q.1- Écrire la fonction **fibo (n)** qui reçoit en paramètre un entier positif n , et qui retourne la valeur du terme F_n de la suite *Fibonacci*, en utilisant l'algorithme ci-dessus.

Q.2- Écrire la fonction **liste_fibo (k)** qui reçoit en paramètre un entier positif k , et qui retourne une liste F contenant les k premiers termes de la suite de Fibonacci : $[F_0, F_1, F_2, \dots, F_{k-2}, F_{k-1}]$

Q.3- Écrire la fonction **produit (L)** qui reçoit en paramètre une liste L de nombres, et qui retourne la valeur du produit des éléments de L .

Q.4- Déterminer la complexité de la fonction **produit (L)**, avec justification.

Q.5- Écrire la fonction **prod_fibo (k)** qui reçoit en paramètre un entier positif k , et qui retourne la valeur du produit des k premiers termes de la suite de Fibonacci : $F_0 * F_1 * F_2 * \dots * F_{k-2} * F_{k-1}$

Q.6- Écrire la fonction **som_fibo (L)** qui reçoit en paramètre une liste L de nombres entiers positifs. La fonction retourne la somme des fibo(x) pour tout x élément de L .

Exemple : **som_fibo ([5, 12, 0, 8])** retourne la valeur de $F_5 + F_{12} + F_0 + F_8$

Partie I : PROGRAMMATION ET CALCUL SCIENTIFIQUE***Produit d'une chaîne matricielle***

Dans cette partie, on suppose que le module **numpy** est importé :

```
import numpy as np
```

Le produit matriciel d'une chaîne de matrices (séquence d'au moins deux matrices) est un problème d'optimisation qui permet de trouver le moyen le plus efficace pour calculer le produit de plusieurs matrices.

La multiplication matricielle est associative, car peu importe comment le produit est mis entre parenthèses, le résultat obtenu restera le même. Par exemple, pour quatre matrices M_0 de dimension **(20, 10)** (20 lignes et 10 colonnes), M_1 de dimension **(10, 35)**, M_2 de dimension **(35, 8)**, et M_3 de dimension **(8, 12)**, on a :

$$((M_0 * M_1) * M_2) * M_3 = (M_0 * (M_1 * M_2)) * M_3 = (M_0 * M_1) * (M_2 * M_3) = M_0 * ((M_1 * M_2) * M_3) = M_0 * (M_1 * (M_2 * M_3))$$

Cependant, l'ordre dans lequel le produit est mis entre parenthèses affecte le nombre de multiplications nécessaires pour calculer ce produit. Le nombre de multiplications dans chaque produit est présenté dans le tableau suivant :

Produit matriciel	Compte des multiplications
$((M_0 * M_1) * M_2) * M_3$	$((20 * 10 * 35) + 20 * 35 * 8) + 20 * 8 * 12 = 14520$
$(M_0 * (M_1 * M_2)) * M_3$	$(10 * 35 * 8 + (20 * 10 * 8)) + 20 * 8 * 12 = 6320$
$(M_0 * M_1) * (M_2 * M_3)$	$(20 * 10 * 35) + (35 * 8 * 12) + 20 * 35 * 12 = 18760$
$M_0 * ((M_1 * M_2) * M_3)$	$10 * 35 * 8 + ((10 * 8 * 12) + 20 * 10 * 12) = 6160$
$M_0 * (M_1 * (M_2 * M_3))$	$35 * 8 * 12 + (10 * 35 * 12 + (20 * 10 * 12)) = 9960$

De toute évidence, le produit $M_0 * ((M_1 * M_2) * M_3)$ est le plus efficace.

Énoncé du problème : On se donne une chaîne de n matrices $M_0, M_1, M_2, \dots, M_{n-1}$ (avec $n > 1$), et on souhaite calculer le produit matriciel : $M_0 * M_1 * M_2 * \dots * M_{n-1}$ (on suppose que toutes les matrices ont des tailles compatibles, c'est-à-dire que le produit est bien défini). Le produit matriciel étant associatif, n'importe quel « parenthésage » du produit donnera le même résultat. Avant d'effectuer tout calcul, on cherche à déterminer quel « parenthésage » nécessitera le moins de multiplications.

Pour représenter la chaîne de matrices $M_0, M_1, \dots, M_{n-1}$, on utilise une liste $T = [t_0, t_1, t_2, \dots, t_n]$, telle que :

- t_0 est le nombre de ligne dans la matrice M_0
- t_1 est à la fois le nombre de colonnes dans M_0 , et le nombre de lignes dans M_1
- t_2 est à la fois le nombre de colonnes dans M_1 , et le nombre de lignes dans M_2
- ...
- t_{n-1} est à la fois le nombre de colonnes dans M_{n-2} , et le nombre de lignes dans M_{n-1}
- t_n est le nombre de colonnes dans M_{n-1}

Exemple :

$T = [20, 10, 35, 8, 12]$

La liste T représente la chaîne de matrices dont les tailles sont : **(20, 10)**, **(10, 35)**, **(35, 8)** et **(8, 12)**.

A- Calcul du nombre minimum de multiplications dans le produit d'une chaîne matricielle

A.1- Algorithme naïf

Une solution possible est de procéder par force brute en énumérant tous les « parenthésages » possibles pour en retenir le meilleur. L'idée est de décomposer le problème en un ensemble de sous-problèmes liés.

Pour calculer le nombre minimal de multiplications dans le produit matriciel de la chaîne de matrice $M_0, M_1, M_2, \dots, M_{n-1}$:

- On découpe cette séquence de matrices en deux séquences : $M_0, M_1, \dots, M_k$ et $M_{k+1}, M_{k+2}, \dots, M_{n-1}$;
- En utilisant la récursivité, on calcule m_1 et m_2 les deux nombres minimaux de multiplications dans le produit de chacune des deux séquences ;
- À la somme m_1+m_2 , on ajoute le nombre de multiplications dans le produit des deux matrices résultats des deux séquences ;
- Faire cela pour chaque position possible k à laquelle la séquence de matrices peut être découpée, et prendre le minimum sur toutes.

Voici l'algorithme d'une fonction récursive qui calcul le nombre minimal de multiplications dans le produit d'une chaîne de matrices, représentée par la liste T . i et j sont deux indices dans la liste T tels que $i < j$.

Fonction $\text{minProduit}(T, i, j)$

Si $i = j-1$ Alors

retourner 0

Fin Si

$R \leftarrow$ liste vide

Pour $k = i+1$ à $j-1$ faire

*$c \leftarrow \text{minProduit}(T, i, k) + \text{minProduit}(T, k, j) + T[i]*T[k]*T[j]$*

Ajouter c à la liste R

Fin Pour

retourner le minimum de R

Q.7- Écrire la fonction **$\text{minProduit}(T, i, j)$** qui reçoit en paramètres une liste T représentant une chaîne de matrices, i et j deux indices dans T tels que $i < j$. La fonction retourne le nombre minimum de multiplications, dans le produit de la chaîne de matrices représentée par la liste T .

Exemple :

$T = [20, 10, 35, 8, 12]$

$\text{minProduit}(T, 0, \text{len}(T)-1)$ retourne le nombre : **6160**

A.2- Programmation dynamique Top-Down (Mémoïsation)

La fonction précédente **minProduit** n'est pas envisageable, sa complexité est exponentielle (elle fait appel à elle-même, plusieurs fois, avec les mêmes paramètres). Pour cela, on propose d'utiliser la mémoïsation :

On utilise un dictionnaire D , dans lequel on stocke les valeurs renvoyées par les appels de la fonction. Et lorsque la fonction est appelée à nouveau avec les mêmes paramètres, elle renvoie la valeur stockée dans le dictionnaire D , au lieu de la recalculer.

- Les clés du dictionnaire D sont les différents tuples d'indices (i, j) tels que $0 \leq i < j < \text{taille de } T$;
- La valeur de chaque clé (i, j) de D est le nombre minimum de multiplications dans le produit de la chaîne de matrices représentée par la sous liste $[T_i, T_{i+1}, \dots, T_j]$.

Q.8- Écrire la fonction **minPrd** (T, i, j) qui reçoit en paramètres une liste T représentant une chaîne de matrices, i et j deux indices dans T tels que $i < j$. En utilisant le principe de la mémorisation, la fonction retourne le nombre minimal de multiplications dans le produit de la chaîne de matrices représentée par T .

Exemple :

$D = \{\}$ et $T = [20, 10, 35, 8, 12]$

minPrd ($T, 0, \text{len}(T)-1$) retourne le nombre : **6160**

Le dictionnaire D contient les éléments suivants :

$\{(0, 1): 0, (1, 2): 0, (2, 3): 0, (3, 4): 0, (2, 4): 3360, (1, 3): 2800, (1, 4): 3760, (0, 2): 7000, (0, 3): 4400, (0, 4): 6160\}$

A.3- Programmation dynamique Bottom-Up

Le problème a une structure telle qu'un « sous-parenthésage » optimal est lui-même optimal. De plus un même « sous-parenthésage » peut intervenir dans plusieurs « parenthésages » différents. Ces deux conditions rendent possible la mise en œuvre de la programmation dynamique.

On remarque que pour un parenthésage optimal du produit $M_i * M_{i+1} \dots * M_j$, si le dernier produit matriciel calculé est $(M_i * M_{i+1} \dots * M_k) * (M_{k+1} * M_{k+2} \dots * M_j)$, alors les « parenthésages » utilisés pour le calcul des produits $M_i * M_{i+1} \dots * M_k$ et $M_{k+1} * M_{k+2} \dots * M_j$ sont eux aussi optimaux.

La même hypothèse d'optimalité peut être faite pour tous les « parenthésages » de tous les produits intermédiaires au calcul du produit $M_i * M_{i+1} \dots * M_j$ et donc pour tous ceux du calcul du produit $M_0 * M_1 * M_2 \dots * M_{n-1}$. Cela permet une résolution grâce à la programmation dynamique.

Pour calculer le nombre minimal de multiplication des « sous-parenthésages », on utilise une matrice carrées C de n lignes et n colonnes (n étant le nombre de matrices dans le produit), telle que chaque élément $C_{i,j}$ contient le nombre minimal de multiplications dans le produit matriciel $M_i * M_{i+1} \dots * M_j$

Pour calculer les éléments de la matrice C , on utilise l'algorithme (1) suivant :

$$(1) \quad \begin{cases} \text{si } i \geq j \text{ alors } C_{i,j} \leftarrow 0 \\ \text{Sinon } C_{i,j} \leftarrow \min_{i \leq k < j} \{C_{i,k} + C_{k+1,j} + T_i * T_{k+1} * T_{j+1}\} \end{cases}$$

Exemple :

$T = [20, 10, 35, 8, 12]$

La matrice C contient les éléments suivants :

[	0	7000	4400	6160	]
[	0	0	2800	3760	]
[	0	0	0	3360	]
[	0	0	0	0	]

NB : L'élément en haut à droite de la matrice C contient le nombre minimal de multiplications.

Les éléments de **C** doivent être calculés de proche en proche, dans l'ordre des diagonales suivant :

Matrice C	
Première diagonale	7000 → 2800 → 3360
Deuxième diagonale	4400 → 3760
Troisième diagonale	6160

Q.9- Écrire la fonction **matriceC (T)** qui reçoit en paramètre une liste **T** représentant une chaîne de matrices. En utilisant l'algorithme (1) décrit précédemment, la fonction retourne la matrice **C**.

B- Recherche du « parenthésage » optimal

Pour trouver le « parenthésage » optimal qui correspond au nombre minimal de multiplications dans le produit de la chaîne de matrices, on utilise une matrice carrée **P** de même dimension que **C**, telle que chaque élément **P_{i,j}** contient l'indice **k** tel que le « parenthésage » optimal du produit soit **(M_i*M_{i+1}*...*M_k) * (M_{k+1}*M_{k+2}*...*M_j)**.

Pour calculer les éléments de la matrice **P**, on utilise l'algorithme (2) suivant :

$$(2) \quad \begin{cases} \text{si } i \geq j \text{ alors } P_{i,j} \leftarrow 0 \\ \text{Sinon } P_{i,j} \leftarrow k, \text{ tel que } C_{i,k} + C_{k+1,j} + T_i * T_{k+1} * T_{j+1} = C_{i,j} \end{cases}$$

Exemple :

T = [20, 10, 35, 8, 12]

Les matrices **C** et **P** sont les suivantes :

Matrice C	Matrice P
[[0 7000 4400 6160]	[[0 0 0 0]
[0 0 2800 3760]	[0 0 1 2]
[0 0 0 3360]	[0 0 0 2]
[0 0 0 0]]	[0 0 0 0]]

Les éléments de **P** doivent être calculés de proche en proche, dans l'ordre des diagonales suivant :

	Matrice C	Matrice P
Première diagonale	7000 → 2800 → 3360	0 → 1 → 2
Deuxième diagonale	4400 → 3760	0 → 2
Troisième diagonale	6160	0

Q.10- Écrire la fonction **matriceP (T, C)** qui reçoit en paramètres une liste **T** représentant la chaîne de matrices et la matrice **C** résultant de l'algorithme (1). En utilisant l'algorithme (2) décrit précédemment, la fonction retourne la matrice **P**.

C- Calcul du produit matricielle d'une chaîne de matrices

Chaque élément $P_{i,j}$ de la matrice P , contient l'indice k tel que le « parenthésage » optimal du produit matriciel $M_i * M_{i+1} * \dots * M_k * M_{k+1} * M_{k+2} * \dots * M_j$ soit $(M_i * M_{i+1} * \dots * M_k) * (M_{k+1} * M_{k+2} * \dots * M_j)$. Donc on peut utiliser les éléments de la matrice P pour calculer le produit d'une chaîne matricielle.

Exemple :

On suppose que la liste M contient les matrices d'une chaîne matricielle : $M = [M_0, M_1, M_2, M_3]$

Et la matrice P est la suivante :

$$P = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

À partir de la matrice P , on peut déduire que :

- ✓ Le « parenthésage » optimal du produit $M_0 * M_1 * M_2 * M_3$ est $M_0 * (M_1 * M_2 * M_3)$ car $P_{0,3}$ vaut 0
- ✓ Le « parenthésage » optimal du produit $M_1 * M_2 * M_3$ est $(M_1 * M_2) * M_3$ car $P_{1,3}$ vaut 2

Ainsi, pour calculer le **produit** matriciel de la chaîne matricielle M , on peut utiliser la relation de récurrence suivante :

i et j sont deux indices dans M tels que $i \leq j$.

- si $i=j$ alors **Produit** (M, i, j) = M_i
- si $i=j-1$ alors **Produit** (M, i, j) = $M_i * M_j$
- si $i < j-1$ alors **Produit** (M, i, j) = **Produit** ($M, i, P_{i,j}$) * **Produit** ($M, P_{i,j} + 1, j$)

Rappel :

Le module **numpy** contient la méthode **dot**, qui permet de calculer le produit matriciel de deux matrices.

Q.11- Écrire la fonction **produit** (M, i, j, P) qui reçoit en paramètres la liste M des matrices, i et j deux indices dans M tels que $i \leq j$, et la matrice P résultat de l'algorithme (2). La fonction calcule le produit matriciel de la chaîne de matrices M , en utilisant le « parenthésage » optimal, et elle retourne la matrice résultante.

Exemple :

$$M = [M_0, M_1, M_2, M_3].$$

La fonction **produit** ($M, 0, \text{len}(M)-1, P$) retourne la matrice résultat du produit matriciel de la chaîne de matrices M , en utilisant le « parenthésage » optimal : $M_0 * (M_1 * M_2) * M_3$

Q.12- Écrire la fonction **calcul** (M) qui reçoit en paramètre la liste M contenant au moins deux matrices compatibles pour le produit matriciel. La fonction calcule le produit matriciel des matrices de M , en utilisant le « parenthésage » optimal, et elle retourne la matrice résultante.

Exemple :

$$M = [M_0, M_1, M_2, M_3].$$

La fonction **calcul** (M) retourne la matrice résultat du produit matriciel de la chaîne de matrices M , en utilisant le « parenthésage » optimal : $M_0 * (M_1 * M_2) * M_3$

Partie II : Fleurs d'Iris

Les fleurs d'*Iris* sont des plantes vivaces à rhizomes ou à bulbes de la famille des Iridacées. On trouve plusieurs espèces de fleurs d'Iris, réparties en plusieurs genres.



A- Base de données des fleurs d'Iris

On considère une base de données composée de deux tables : '**Espèces**' et '**Fleurs**'.

<u><i>Espèces</i></u>	
code	(texte)
nom	(texte)
genre	(texte)

<u><i>Fleurs</i></u>	
codF	(entier)
long_sépale	(réel)
larg_sépale	(réel)
long_pétale	(réel)
larg_pétale	(réel)
codEsp	(texte)

Structure de la table '**Espèces**'

La table '**Espèces**' contient des informations sur les espèces des fleurs d'iris, et le genre de chaque espèce. Cette table est composée de **trois** champs :

- ✓ Le champ **code** contient un texte unique permettant d'identifier chaque espèce ;
- ✓ Le champ **nom** contient le nom de chaque espèce ;
- ✓ Le champ **genre** contient le genre de chaque espèce.

Exemples :

<i>code</i>	<i>nom</i>	<i>genre</i>
<i>vrs</i>	<i>Versicolor</i>	<i>Limniris</i>
<i>vrg</i>	<i>Virginica</i>	<i>Limniris</i>
<i>sts</i>	<i>Setosa</i>	<i>Limniris</i>
<i>srt</i>	<i>Serotina</i>	<i>Xiphium</i>
<i>lat</i>	<i>Latifolia</i>	<i>Xiphium</i>
...	...	...

Structure de la table 'Fleurs'

La table '**Fleurs**' contient des informations sur plusieurs fleurs. Cette table est composée de **six** champs :

- ✓ Le champ **codF** contient un entier unique permettant d'identifier chaque fleur ;
- ✓ Le champ **long_sépale** contient la longueur du sépale de la fleur ;
- ✓ Le champ **larg_sépale** contient la largeur du sépale de la fleur ;
- ✓ Le champ **long_pétale** contient la longueur du pétale de la fleur ;
- ✓ Le champ **larg_pétale** contient la largeur du pétale de la fleur ;
- ✓ Le champ **codEsp** contient le code de l'espèce de la fleur.

Exemples :

<i>codF</i>	<i>long_sépale</i>	<i>larg_sépale</i>	<i>long_pétale</i>	<i>larg_pétale</i>	<i>codEsp</i>
1	7.0	3.2	4.7	1.4	<i>vrs</i>
2	6.4	2.9	4.3	1.3	<i>vrs</i>
3	5.9	3.0	4.2	1.5	<i>vrs</i>
4	5.4	3.4	1.7	0.2	<i>sts</i>
5	4.4	3.2	1.3	0.2	<i>sts</i>
...	...	...	...	...	...

Q.13 – Déterminer les *clés primaires* et les *clés étrangères* dans cette base de données.

Q.14 – Écrire en algèbre relationnelle, la requête qui donne pour résultat *les codes, les longueurs des sépales, les largeurs des sépales, les longueurs des pétales et les largeurs des pétales des fleurs de l'espèce de nom 'Marsica' et des fleurs de l'espèce de nom 'Latifolia'*.

Q.15 – Écrire en langage SQL, la requête qui donne pour résultat *le compte des différents genres*.

Q.16 – Écrire en langage SQL, la requête qui donne pour résultat *les codes, les longueurs des sépales, les largeurs des sépales, les longueurs des pétales et les largeurs des pétales des fleurs du genre 'Scorpiris'*. Le résultat doit être trié dans l'ordre croissant des codes des fleurs.

Q.17 – Écrire en langage SQL, la requête qui donne pour résultat *pour chaque genre, le compte des espèces, le compte des fleurs, la plus petite et la plus grande longueur des sépales. La moyenne des largeurs des pétales doit être supérieure à 2.026. Le résultat doit être trié dans l'ordre décroissant de la somme des longueurs des pétales*.

B- Apprentissage automatique : Classification des fleurs d'Iris

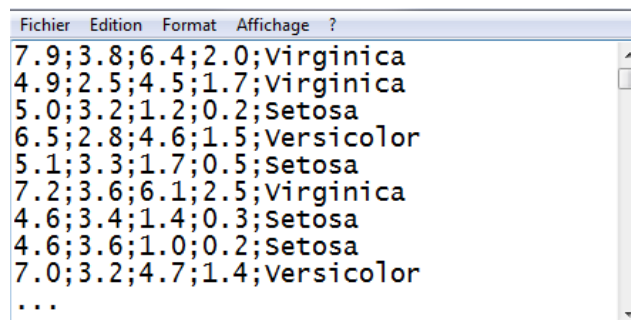
L'apprentissage automatique (en anglais : *machine learning*), est un champ d'étude de l'intelligence artificielle qui se fonde sur des approches mathématiques et statistiques pour donner aux ordinateurs la capacité d'« apprendre » à partir de données, c'est-à-dire d'améliorer leurs performances à résoudre des tâches sans être explicitement programmés pour chacune. Les algorithmes d'apprentissage peuvent se catégoriser selon le mode d'apprentissage qu'ils emploient.

Si les classes des données sont prédéterminées et les exemples connus, le système apprend à classer selon un modèle de classification ou de classement ; on parle alors d'apprentissage supervisé.

Dans la suite de cette partie, on se propose de réaliser la classification d'une fleur d'iris, en utilisant un fichier texte contenant plusieurs échantillons de fleurs d'iris de différentes espèces (Setosa, Versicolor, Virginica, ...)

On dispose d'un fichier texte regroupant les caractéristiques de plusieurs espèces de fleurs d'iris. Dans ce fichier, chaque ligne représente une observation de 5 caractéristiques (séparées par ;) d'une fleur d'iris : la **longueur** et la **largeur** de ses sépales, la **longueur** et la **largeur** de ses pétales et son **espèce**.

Exemple :



Lecture des données

Q.18- Écrire la fonction **convertir (S)** qui reçoit en paramètre une chaîne de caractères **S**, représentant une ligne dans le fichier texte. La fonction retourne un tuple contenant les mesures de la longueur et la largeur des sépales, et la longueur et la largeur des pétales d'une fleur d'iris.

Exemples :

- **convertir ('7.9;3.8;6.4;2.0;Virginica')** retourne la liste suivante : **[7.9, 3.8, 6.4, 2.0, 'Virginica']**
- **convertir ('5.0;3.2;1.2;0.2;Setosa')** retourne la liste suivante : **[5.0, 3.2, 1.2, 0.2, 'Setosa']**
- **convertir ('6.5;2.8;4.6;1.5;Versicolor')** retourne la liste suivante : **[6.5, 2.8, 4.6, 1.5, 'Versicolor']**

Q.19- Écrire la fonction **lire_fichier (ch)** qui reçoit en paramètre une chaîne de caractères **ch** qui représente le chemin du fichier texte des fleurs d'iris. La fonction retourne une liste **D** de sous-listes, telles que chaque sous-liste contient les mesures de la longueur et la largeur des sépales, la longueur et la largeur des pétales et l'espèce d'une fleur d'iris.

Exemple :

lire_fichier ('../fleurs_iris.txt') retourne la liste **D** des fleurs suivante :

D = [[7.9, 3.8, 6.4, 2.0, 'Virginica'], [4.9, 2.5, 4.5, 1.7, 'Virginica'], [5.0, 3.2, 1.2, 0.2, 'Setosa'], [6.5, 2.8, 4.6, 1.5, 'Versicolor'], [5.1, 3.3, 1.7, 0.5, 'Setosa'], [7.2, 3.6, 6.1, 2.5, 'Virginica'], [4.6, 3.4, 1.4, 0.3, 'Setosa'], [4.6, 3.6, 1.0, 0.2, 'Setosa'], [7.0, 3.2, 4.7, 1.4, 'Versicolor'], [6.7, 3.3, 5.7, 2.5, 'Virginica'], ...]

Algorithme des KNN

L'algorithme des **k plus proches voisins** (*KNN : K Nearest Neighbors*) est un algorithme relativement simple, très utilisé pour effectuer des classifications. Il ne présente pas de phase d'entraînement, il utilise toutes les données à chaque calcul.

Pour prédire l'espèce d'une nouvelle fleur **F**, le principe de l'algorithme **KNN** est le suivant :

- Définir une fonction qui calcule la distance entre deux fleurs ;
- Choisir un nombre entier strictement positif **k** ;
- Calculer toutes les distances entre la fleur **F** et toutes les autres fleurs de **D** ;
- Créer la liste **V** des **k** fleurs de **D** les proches à **F** selon la fonction de calcul de distance ;
- Retourner l'espèce la plus dominante dans la liste **V**, comme étant l'espèce prédite pour la fleur **F**.

Distance entre deux fleurs

La plupart des techniques de classification utilisent la distance euclidienne comme mesure de similarité.

F étant une liste qui représente une nouvelle fleur dont on cherche à prédire l'espèce, et **X** étant une fleur de la liste **D** des fleurs dont l'espèce est connue. La distance entre les deux fleurs **F** et **X** est définie comme la racine carrée de la somme des différences au carré de chacun de leurs mesures des sépales et des pétales. Elle est exprimée par la formule suivante :

$$\text{distance}(F, X) = \sqrt{\sum_{i=0}^3 (F_i - X_i)^2}$$

Q.20- Écrire la fonction **distance(F, X)** qui reçoit en paramètres les deux fleurs **F** et **X** et qui retourne la distance euclidienne entre les deux fleurs **F** et **X**.

Exemples :

- **distance([5.0, 3.5, 1.6, 0.6], [7.9, 3.8, 6.4, 2.0, 'Virginica'])** retourne **5.787**
- **distance([6.8, 2.8, 4.8, 1.4], [6.5, 2.8, 4.6, 1.5, 'Versicolor'])** retourne **0.374**

Q.21- On considère que la racine carrée est de complexité constante. Justifier que la fonction **distance** est de complexité constante.

Tri de la liste des fleurs

Q.22- Écrire la fonction de complexité quasi-linéaire **tri(D, F)** qui reçoit en paramètres la liste des fleurs **D** et une nouvelle fleur **F** dont l'espèce est inconnue. La fonction trie les fleurs de **D** dans l'ordre croissant de la distance entre la fleur **F** et chaque fleur de **D**.

Exemple :

Après l'appel de la fonction **tri(D, [6.4, 3.0, 4.6, 1.9])**, on obtient la liste suivante :

[[6.1, 3.0, 4.9, 1.8, 'Virginica'], [6.3, 2.7, 4.9, 1.8, 'Virginica'], [6.5, 2.8, 4.6, 1.5, 'Versicolor'], [6.4, 3.2, 4.5, 1.5, 'Versicolor'], [6.0, 3.0, 4.8, 1.8, 'Virginica'], [6.7, 3.1, 4.7, 1.5, 'Versicolor'], [6.6, 3.0, 4.4, 1.4, 'Versicolor'], [6.0, 2.9, 4.5, 1.5, 'Versicolor'], [5.9, 3.2, 4.8, 1.8, 'Versicolor'], [6.1, 2.9, 4.7, 1.4, 'Versicolor'], [6.5, 3.0, 5.2, 2.0, 'Virginica'], [6.7, 3.1, 4.4, 1.4, 'Versicolor'], ...]

Liste des k plus proches fleurs voisines

Q.23- Écrire la fonction **proches** (F , D , k) qui reçoit en paramètres une nouvelle fleur F dont l'espèce est inconnue, la liste des fleurs D triée dans l'ordre croissant de la distance entre F et chaque fleur de D , et k un entier strictement positif. La fonction retourne une nouvelle liste V qui contient les k fleurs de D les plus proches à la fleur F selon la distance euclidienne.

Exemples :

- **proches** ([6.4, 3.0, 4.6, 1.9], D , 5) retourne la liste V suivante : [[6.2, 2.8, 4.8, 1.8, 'Virginica'], [6.1, 3.0, 4.9, 1.8, 'Virginica'], [6.3, 3.3, 4.7, 1.6, 'Versicolor'], [6.3, 2.7, 4.9, 1.8, 'Virginica'], [6.5, 2.8, 4.6, 1.5, 'Versicolor']]
- **proches** ([6.4, 3.0, 4.6, 1.9], D , 9) retourne la liste V suivante : [[6.2, 2.8, 4.8, 1.8, 'Virginica'], [6.1, 3.0, 4.9, 1.8, 'Virginica'], [6.3, 3.3, 4.7, 1.6, 'Versicolor'], [6.3, 2.7, 4.9, 1.8, 'Virginica'], [6.5, 2.8, 4.6, 1.5, 'Versicolor'], [6.4, 3.2, 4.5, 1.5, 'Versicolor'], [6.0, 3.0, 4.8, 1.8, 'Virginica'], [6.7, 3.1, 4.7, 1.5, 'Versicolor'], [6.7, 3.0, 5.0, 1.7, 'Versicolor']]

Occurrences des espèces des k plus proches fleurs voisines

Q.24- Écrire la fonction **occurrences** (V) qui reçoit en paramètre la liste V des plus proches fleurs voisines, et qui retourne un dictionnaire R tel que :

- Les clés du dictionnaire R sont les différentes espèces des fleurs de V ;
- La valeur de chaque clé du dictionnaire R est l'occurrence de cette espèce dans les fleurs de V .

Exemples :

- $V = [[6.2, 2.8, 4.8, 1.8, 'Virginica'], [6.1, 3.0, 4.9, 1.8, 'Virginica'], [6.3, 3.3, 4.7, 1.6, 'Versicolor'], [6.3, 2.7, 4.9, 1.8, 'Virginica'], [6.5, 2.8, 4.6, 1.5, 'Versicolor']]$
occurrences (V) retourne le dictionnaire $R = \{ 'Virginica' : 3, 'Versicolor' : 2 \}$
- $V = [[6.2, 2.8, 4.8, 1.8, 'Virginica'], [6.1, 3.0, 4.9, 1.8, 'Virginica'], [6.3, 3.3, 4.7, 1.6, 'Versicolor'], [6.3, 2.7, 4.9, 1.8, 'Virginica'], [6.5, 2.8, 4.6, 1.5, 'Versicolor'], [6.4, 3.2, 4.5, 1.5, 'Versicolor'], [6.0, 3.0, 4.8, 1.8, 'Virginica'], [6.7, 3.1, 4.7, 1.5, 'Versicolor'], [6.7, 3.0, 5.0, 1.7, 'Versicolor']]$
occurrences (V) retourne le dictionnaire $R = \{ 'Virginica' : 4, 'Versicolor' : 5 \}$

Classification d'une nouvelle fleur

Q.25- Écrire la fonction **knn** (F , D , k) qui reçoit en paramètres une nouvelle fleur F dont l'espèce est inconnue, la liste D des fleurs dont les espèces sont connues et un entier strictement positif k . En utilisant l'algorithme **KNN**, la fonction retourne le nom de l'espèce prédite pour la fleur F .

Exemples :

- **knn** ([6.4, 3.0, 4.6, 1.9], D , 5) retourne l'espèce 'Virginica'
- **knn** ([6.4, 3.0, 4.6, 1.9], D , 9) retourne l'espèce 'Versicolor'
- **knn** ([5.1, 3.0, 1.9, 0.5], D , 8) retourne l'espèce 'setosa'

Test du model

Notre modèle est capable de prédire l'espèce d'une nouvelle fleur d'iris. Pour le tester, on propose de l'appliquer sur plusieurs fleurs d'iris, dont les espèces sont connues. Et de comparer l'espèce effective de chaque fleur avec le résultat de prédiction du modèle.

On suppose que la liste **T** contient plusieurs listes qui représentent des fleurs d'iris de différentes espèces.

Exemple :

T = [[5.0, 3.0, 1.6, 0.2, 'Setosa'], [4.9, 3.6, 1.4, 0.1, 'Setosa'], [5.1, 3.7, 1.5, 0.4, 'Setosa'], [5.7, 2.5, 5.0, 2.0, 'Virginica'], [6.3, 3.3, 4.7, 1.6, 'Versicolor'], [6.3, 2.5, 4.9, 1.5, 'Versicolor'], [6.1, 3.0, 4.6, 1.4, 'Versicolor'], [6.2, 2.8, 4.8, 1.8, 'Virginica'], [5.2, 2.7, 3.9, 1.4, 'Versicolor'], [5.8, 2.7, 5.1, 1.9, 'Virginica'], [5.6, 2.9, 3.6, 1.3, 'Versicolor'], [5.7, 2.9, 4.2, 1.3, 'Versicolor'], [6.2, 2.2, 4.5, 1.5, 'Versicolor'], ...]

Q.26- Écrire la fonction **test (F, D, k)** qui reçoit en paramètres une liste représentant une nouvelle fleur **F** de la liste **T**, la liste des fleurs **D**, et **k** un entier strictement positif. En utilisant l'algorithme **KNN**, la fonction effectue la classification de **F** :

- Si la classification KNN de **F** donne l'espèce effective de **F**, alors la fonction retourne **True**
- Sinon, la fonction retourne **False**.

Exemples :

- **test ([5.0, 3.0, 1.6, 0.2, 'Setosa'], D, 5)** retourne **True**
- **test ([6.7, 3.0, 5.0, 1.7, 'Versicolor'], D, 7)** retourne **False**

Q.27- Écrire la fonction **compte (T, D, k)** qui reçoit en paramètres la liste des fleurs **T**, la liste des fleurs **D** et **k** un entier strictement positif. La fonction retourne le compte des fleurs de **T** qui ont réussi le test : la classification KNN d'une fleur donne l'espèce effective de cette fleur.

Exemples :

- **compte (T, D, 5)** retourne **49**
- **compte (T, D, 3)** retourne **50**
- **compte (T, D, 9)** retourne **48**

Q.28- Écrire la fonction **pourcentages (T, D, n)** qui reçoit en paramètres la liste des fleurs **T**, la liste des fleurs **D** et **n** un entier supérieur strictement à **1**. La fonction retourne une liste de tuples tels que :

- Les premiers éléments des tuples sont les entiers **k = 2, 3, 4, ..., n**
- Les seconds éléments des tuples sont les pourcentages de tests réussis pour chaque valeur de **k**.

Exemple :

pourcentages (T, D, 10) retourne la liste :

[(2, 94.0), (3, 100.0), (4, 96.0), (5, 98.0), (6, 96.0), (7, 96.0), (8, 96.0), (9, 96.0), (10, 98.0)]

On remarque que pour **k=3**, le pourcentage de tests réussis est de : **100%**

***** **FIN** *****